CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 11

Student Management CRUD Application with Search, Course Filter, Soft Delete and CSV Export

PHP 8 · MySQL (MariaDB) · MySQLi prepared statements

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

1. Aim and Objectives

To implement a complete Student Management application with PHP, MySQL and MySQLi that performs all record operations safely and keeps every record, even after a student is removed from the active list.

- Display student records in a responsive table with a clear **Active / Inactive** status.
- Add and update students with browser and server-side validation, duplicate-email handling and clear messages.
- Search by name, email and course, filter by course (and status), and paginate the list.
- **Soft delete** a student after an explicit confirmation: the status changes from `active` to `inactive` and the row stays in MySQL.
- Export the selected records to a CSV file that opens correctly in Microsoft Excel and is protected against formula injection.
- Use MySQLi prepared statements for every database operation and protect the forms against CSRF, XSS and duplicate submissions.

2. Problem Definition

An institute's student list changes constantly: students join, change course or semester, and leave. Staff must be able to find a student quickly and export the list for spreadsheet work. At the same time a record must never be lost by a wrong click, so removing a student must be reversible at database level – a **soft delete** that only marks the student inactive. Every input comes from the browser and is therefore untrusted: it has to be validated on the server, sent to MySQL only through prepared statements, and escaped whenever it is shown again.

Starting point: the Practical-11 folder already contained a simple version (`index.php`, `edit.php`, `auth.php`, `schema.sql`). It used the shared `student_db` database, required the Practical 10 login database, deleted records permanently through a GET link and changed the status through GET links without CSRF protection. It was replaced by the application described here, which uses its own database `studenthub_prac11` and has no permanent delete. The old files were backed up before the change and remain in the Git history.

3. Tools and Technologies

Tool	Use in this practical
PHP 8.2.12 (XAMPP)	Application pages, validation, sessions, CSV generation
MySQLi	All database access with prepared statements (prepare / bind_param / execute)
MariaDB 10.4.32 (XAMPP)	Database studenthub_prac11; export with mysqldump
HTML5, CSS3, JavaScript	Forms with HTML5 validation, responsive table (style.css), double-click guard (app.js)
Microsoft Edge	Running the application; headless Edge (Playwright) for the browser test and screenshots
Microsoft Excel	Opening the exported CSV file
Python 3	tests/test_app.py – automated end-to-end tests (standard library only)

4. Database Design

The practical uses its own database, `studenthub_prac11`, so the databases of Practicals 8–10 and the main StudentHub application are never touched. It has one InnoDB table, `students`; a second table (for example courses) is not needed, so there are no foreign keys – the five allowed courses are checked by the application.

Column	Type	Notes
student_id	INT UNSIGNED AUTO_INCREMENT	Primary key
full_name	VARCHAR(100) NOT NULL	Index idx_students_full_name
email	VARCHAR(254) NOT NULL	UNIQUE uq_students_email (case-insensitive collation)
course	VARCHAR(60) NOT NULL	Index idx_students_course
semester	TINYINT UNSIGNED NOT NULL	CHECK semester BETWEEN 1 AND 8
status	ENUM('active','inactive')	Default 'active'; index idx_students_status
created_at	DATETIME	Default CURRENT_TIMESTAMP
updated_at	DATETIME	ON UPDATE CURRENT_TIMESTAMP – changes on every update
deleted_at	DATETIME NULL	Time of the soft delete; CHECK: NULL exactly when status = 'active'

`studenthub_prac11.sql` is the mysqldump export of this database: CREATE DATABASE, the table and **24 fictional students** (example.com addresses) – **22 active and 2 inactive**. It can be imported with phpMyAdmin or `mysql -u root < studenthub_prac11.sql`; importing it again resets the table to the seed data.

studenthub_prac11.sql (lines 45–62) – table structure from the dump

```
45  CREATE TABLE `students` (
46    `student_id` int(10) unsigned NOT NULL AUTO_INCREMENT,
47    `full_name` varchar(100) NOT NULL,
48    `email` varchar(254) NOT NULL,
49    `course` varchar(60) NOT NULL,
50    `semester` tinyint(3) unsigned NOT NULL,
51    `status` enum('active','inactive') NOT NULL DEFAULT 'active',
52    `created_at` datetime NOT NULL DEFAULT current_timestamp(),
53    `updated_at` datetime NOT NULL DEFAULT current_timestamp() ON UPDATE current_timestamp(),
54    `deleted_at` datetime DEFAULT NULL,
55    PRIMARY KEY (`student_id`),
56    UNIQUE KEY `uq_students_email` (`email`),
57    KEY `idx_students_course` (`course`),
58    KEY `idx_students_status` (`status`),
59    KEY `idx_students_full_name` (`full_name`),
60    CONSTRAINT `chk_students_semester` CHECK (`semester` between 1 and 8),
61    CONSTRAINT `chk_students_soft_delete` CHECK (`status` = 'active' and `deleted_at` is null or
      `status` = 'inactive' and `deleted_at` is not null)
62  ) ENGINE=InnoDB AUTO_INCREMENT=25 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1 Setting up and running the application

Practical 11 is a standalone application without a login. It needs only XAMPP's PHP and MySQL:

- Start MySQL in the XAMPP Control Panel.
- Import the database: phpMyAdmin → Import → `studenthub_prac11.sql`, or `C:\xampp\mysql\bin\mysql.exe -u root < studenthub_prac11.sql` in the Practical-11 folder.
- Start the server: double-click `start-server.bat`, or run `C:\xampp\php\php.exe -S 127.0.0.1:8611 router.php` in the Practical-11 folder.
- Open **http://127.0.0.1:8611/index.php** in the browser.
- Run the tests: `python tests\test_app.py` (uses the temporary database `studenthub_prac11_test`).

5. Implementation

File	Purpose
index.php	READ – student table, search, course/status filter, pagination (10 per page), messages, Export button
add.php	CREATE – form, validation, duplicate-email check, prepared INSERT

edit.php	UPDATE – form filled from MySQL, validation, prepared UPDATE
delete.php	SOFT DELETE – confirmation page (GET) and status change to inactive (POST)
export.php	CSV export of all records matching the current search and filters
includes/functions.php	Session, security headers, one-time form tokens, messages, validation, search query, layout
db.php	Settings and MySQLi connection; errors are logged, never shown
style.css, app.js	Responsive layout (table becomes cards on phones); disables the button after submitting

5.1 Create and update with validation

Both forms share `student_form()`. The browser checks `required`, `type="email"`, `pattern` and `maxlength` first, but only the server check in `validate_student()` is trusted: name 2–100 letters (spaces, dots, apostrophes and hyphens allowed), valid email (stored in lower case), a course from the list and a semester 1–8. The email must not belong to another student – inactive students included. If anything is wrong the form is shown again with HTTP 422, an error under each field and the typed values. A duplicate inserted at the same moment is also caught by the UNIQUE key (error 1062).

add.php (lines 11–34) – token check, validation and prepared INSERT

```
11  if ($_SERVER['REQUEST_METHOD'] === 'POST') {
12      if (!use_form_token($_POST['form_token'] ?? null)) {
13          reject_invalid_token();
14      }
15      [$values, $errors] = validate_student($_POST);
16      $db = get_db();
17      if (!$errors && email_taken($db, $values['email'])) {
18          $errors['email'] = 'This email is already registered to another student.';
19      }
20      if (!$errors) {
21          try {
22              $stmt = $db->prepare('INSERT INTO students (full_name, email, course, semester) VALUES (?,
    ?, ?, ?)');
23              $stmt->bind_param('sssi', $values['full_name'], $values['email'], $values['course'],
    $values['semester']);
24              $stmt->execute();
25              flash('success', 'Student “' . $values['full_name'] . '” was added successfully (ID ' .
    $stmt->insert_id . ').', $stmt->insert_id);
26              redirect('index.php');
27          } catch (mysqli_sql_exception $exception) {
28              if ($exception->getCode() !== 1062) {               // 1062 = duplicate key (email added
    at the same moment)
29                  throw $exception;
30              }
31              $errors['email'] = 'This email is already registered to another student.';
32          }
33      }
34      http_response_code(422);
```

5.2 Soft delete

The **Deactivate** button only opens `delete.php?id=N`, a confirmation page that shows the student and asks *“Are you sure you want to deactivate …?”*. A GET request never changes data. Only **Yes, deactivate student** sends a POST with a one-time token. The record is not deleted: one prepared UPDATE sets `status = 'inactive'` and `deleted_at = NOW()`, and `AND status = 'active'` makes a second attempt harmless ("already inactive"). The student stays in the list with a red **Inactive** badge, stays in the CSV export and keeps the email reserved. The application contains no DELETE statement.

delete.php (lines 11–25) – confirmed soft delete

```
11  if ($_SERVER['REQUEST_METHOD'] === 'POST') {
12      if (!use_form_token($_POST['form_token'] ?? null)) {
13          reject_invalid_token();
14      }
```

```
15      $id = positive_id($_POST['student_id'] ?? '');
16      $student = $id > 0 ? find_student($db, $id) : null;
17      $stmt = $db->prepare("UPDATE students SET status = 'inactive', deleted_at = NOW() WHERE student_id
     = ? AND status = 'active'");
18      $stmt->bind_param('i', $id);
19      $stmt->execute();
20      if ($stmt->affected_rows === 1) {
21          flash('success', 'Student “' . $student['full_name'] . '” was deactivated. The record is kept
     in the database with status Inactive.', $id);
22      } else {
23          flash('error', $student ? 'Student “' . $student['full_name'] . '” is already inactive.' :
     'That student record was not found.');
24      }
25      redirect('index.php');
```

5.3 Search, course filter and pagination

The search text is matched against name, email and course with `LIKE ?`; `%` and _ typed by the user are escaped so they are searched literally. The course and status values are accepted only if they appear in the fixed lists, so they can never change the SQL. The same WHERE clause is used for counting, for the page of 10 records (`LIMIT ? OFFSET ?`) and for the CSV export; the most recently changed records are listed first.

includes/functions.php (lines 186–207) – WHERE clause with placeholders

```
186  function filter_sql(array $filters): array
187  {
188      $where = [];
189      $types = '';
190      $params = [];
191      if ($filters['q'] !== '') {
192          $like = '%' . addcslashes($filters['q'], '\\%_') . '%';
193          $where[] = '(full_name LIKE ? OR email LIKE ? OR course LIKE ?)';
194          $types .= 'sss';
195          array_push($params, $like, $like, $like);
196      }
197      if ($filters['course'] !== '') {
198          $where[] = 'course = ?';
199          $types .= 's';
200          $params[] = $filters['course'];
201      }
202      if ($filters['status'] !== '') {
203          $where[] = 'status = ?';
204          $types .= 's';
205          $params[] = $filters['status'];
206      }
207      return [$where ? ' WHERE ' . implode(' AND ', $where) : '', $types, $params];
```

5.4 CSV export

`export.php` reads the same filters and writes every matching record (all pages). **Inactive records are included** (Status *Inactive* and a *Deactivated At* time) unless the status filter is set to Active. A UTF-8 byte-order mark, commas and CRLF line endings make Excel open the file directly. Cells that start with `=`, `+`, `-`, `@`, tab or carriage return get a leading apostrophe, so a name such as `=HYPERLINK(...)` is shown as text instead of being run as a formula.

export.php (lines 13–36) – formula-injection protection and CSV output

```
13  function csv_safe(mixed $value): string
14  {
15      $text = (string) $value;
16      return $text !== '' && strpbrk($text[0], "=+-@\t\r") !== false ? "'" . $text : $text;
17  }
18
19  start_app();
20  $filters = list_filters($_GET);
21  $students = fetch_students(get_db(), $filters);           // read everything before sending any CSV
     output
22
23  header('Content-Type: text/csv; charset=utf-8');
24  header('Content-Disposition: attachment; filename="students_' . date('Ymd_His') . '.csv"');
```

```
25
26  $out = fopen('php://output', 'w');
27  fwrite($out, "\xEF\xBB\xBF");
28  fputcsv($out, ['Student ID', 'Full Name', 'Email', 'Course', 'Semester', 'Status', 'Created At',
     'Updated At', 'Deactivated At'],
29          ',', '"', '', "\r\n");
30  foreach ($students as $s) {
31      fputcsv($out, array_map('csv_safe', [
32          $s['student_id'], $s['full_name'], $s['email'], $s['course'], $s['semester'],
33          STATUSES[$s['status']] ?? $s['status'], $s['created_at'], $s['updated_at'], $s['deleted_at']
     ?? '',
34      ]), ',', '"', '', "\r\n");
35  }
36  fclose($out);
```

5.5 Security measures

- **SQL injection:** every query uses `prepare()` with `?` placeholders and `bind_param()`; the test suite also checks that the code contains no `query()` call.
- **XSS:** every value is printed through `e()` = `htmlspecialchars(…, ENT_QUOTES, 'UTF-8')`; a Content-Security-Policy allows only the site's own files.
- **CSRF and duplicate submissions:** each form gets a random 64-character token stored in the session. A POST is accepted only with a token this session issued and has not used yet, so another site cannot submit the forms and a form sent twice (double click, Back + resend) is refused the second time. Add, update and soft delete accept only POST and redirect afterwards (Post/Redirect/Get).
- **Errors:** MySQL errors throw exceptions; a global handler writes the details to the PHP error log and shows only “The student database is not available right now”.
- **Configuration:** the defaults are XAMPP's; local credentials go into the git-ignored `config.local.php`. `includes/`, `tests/`, the SQL dump and the config files are blocked with HTTP 403.

includes/functions.php (lines 60–77) – one-time form tokens

```
60  function form_token(): string
61  {
62      $token = bin2hex(random_bytes(32));
63      $_SESSION['form_tokens'][$token] = time();
64      if (count($_SESSION['form_tokens']) > 50) {                  // keep only the 50 newest open forms
65          $_SESSION['form_tokens'] = array_slice($_SESSION['form_tokens'], -50, null, true);
66      }
67      return $token;
68  }
69
70  function use_form_token(mixed $submitted): bool
71  {
72      if (!is_string($submitted) || !preg_match('/^[0-9a-f]{64}$/', $submitted)
73          || !isset($_SESSION['form_tokens'][$submitted])) {
74          return false;
75      }
76      unset($_SESSION['form_tokens'][$submitted]);
77      return true;
```

6. Output Screenshots

All screenshots are genuine captures of the running application (PHP built-in server, database studenthub_prac11 with the fictional seed data). Figures 1–11 were taken with Microsoft Edge during the browser test; Figure 12 is a normal Edge window after clicking *Export CSV*, and Figure 13 shows that downloaded file opened in Microsoft Excel (columns auto-fitted for readability, the file was closed without saving).

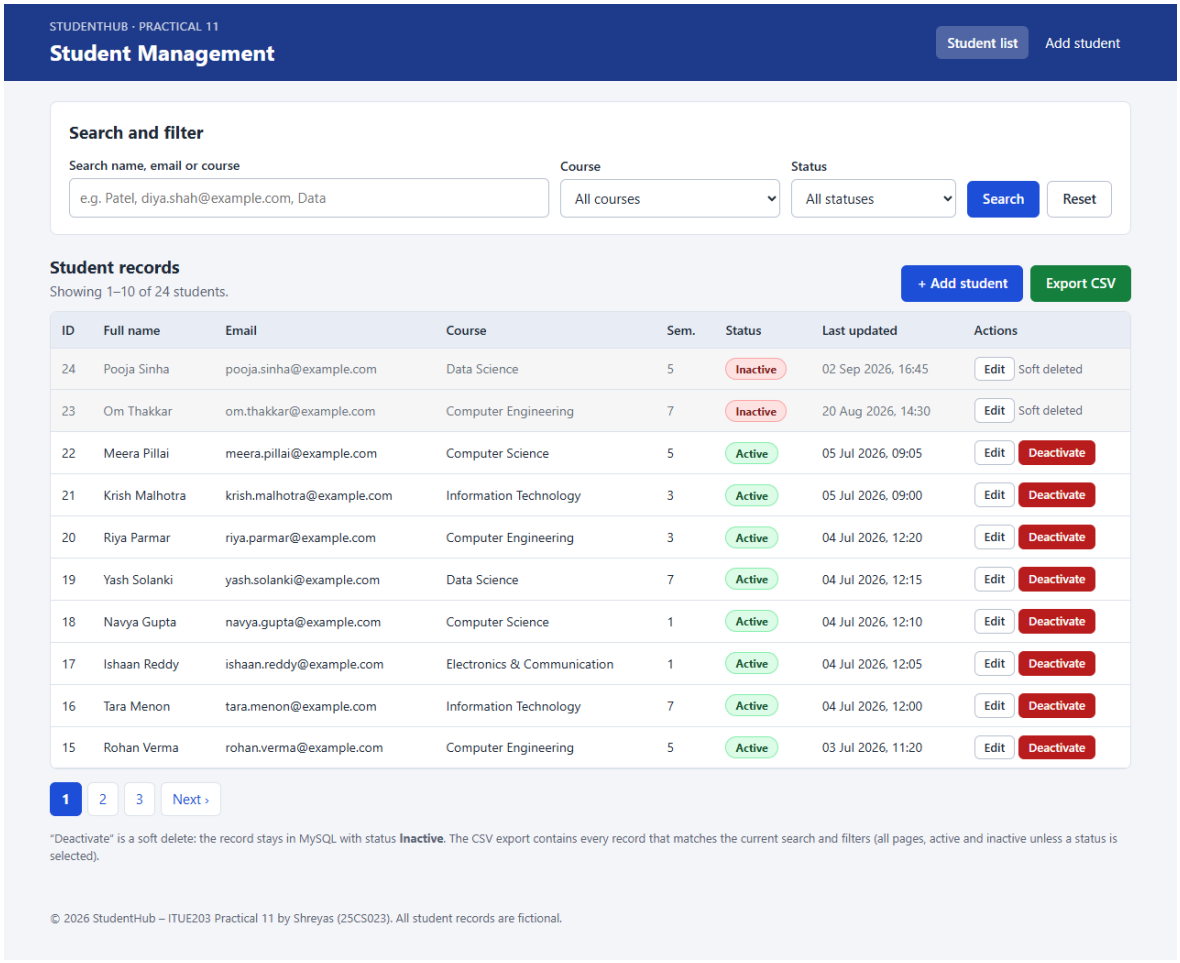


Figure 1: Student records in a table – 24 students, 10 per page, Active/Inactive badges, Edit and Deactivate actions (students-list.png)

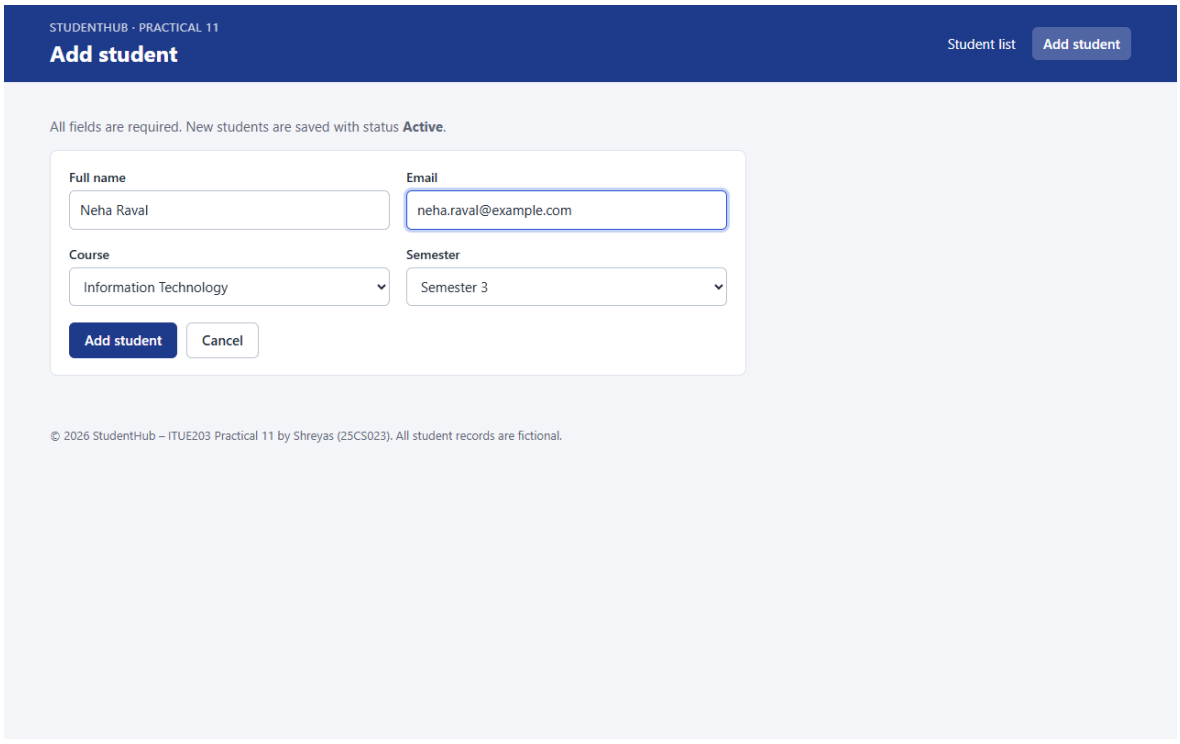


Figure 2: Add student form filled in (Neha Raval, Information Technology, semester 3) (add-student.png)

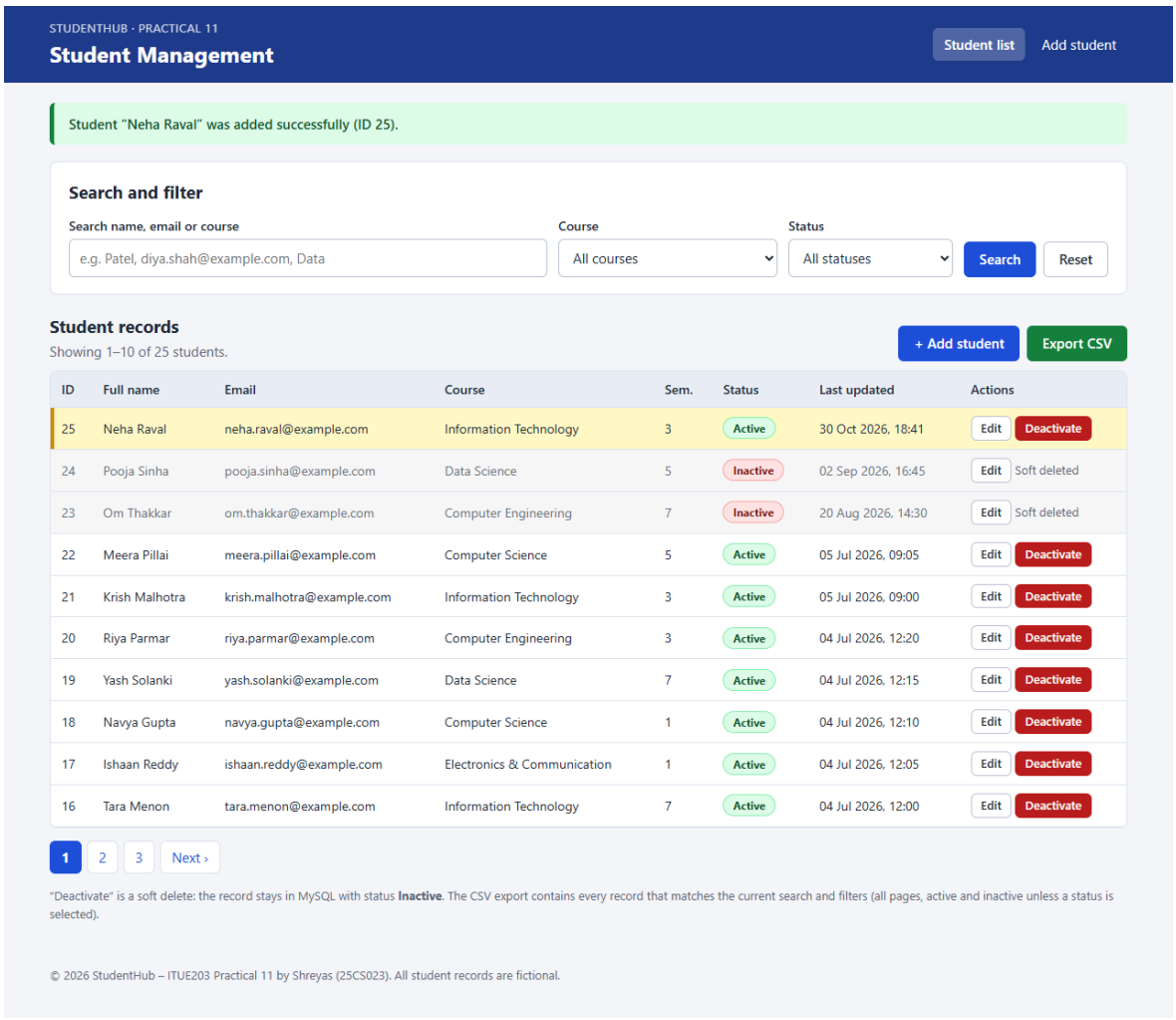


Figure 3: Successful addition – green message and the new student (ID 25) first and highlighted (successful-add.png)

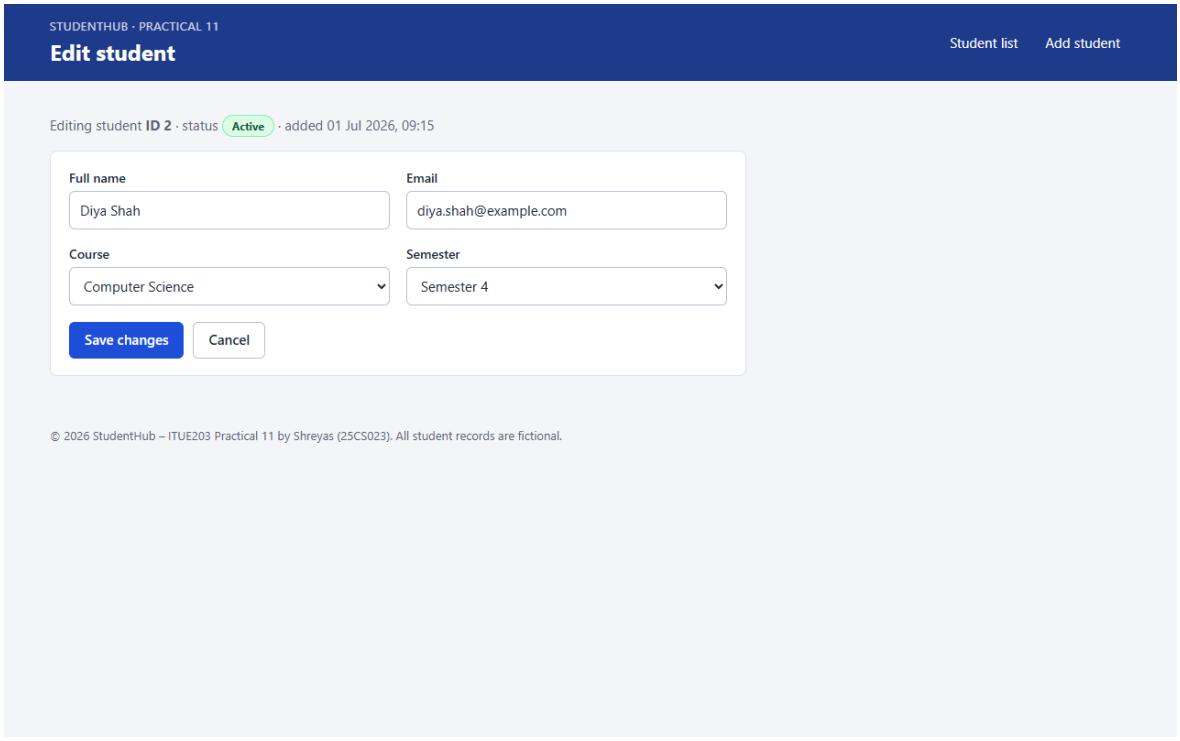


Figure 4: Edit form for Diya Shah (ID 2) – course changed to Computer Science, semester 4 (edit-student.png)

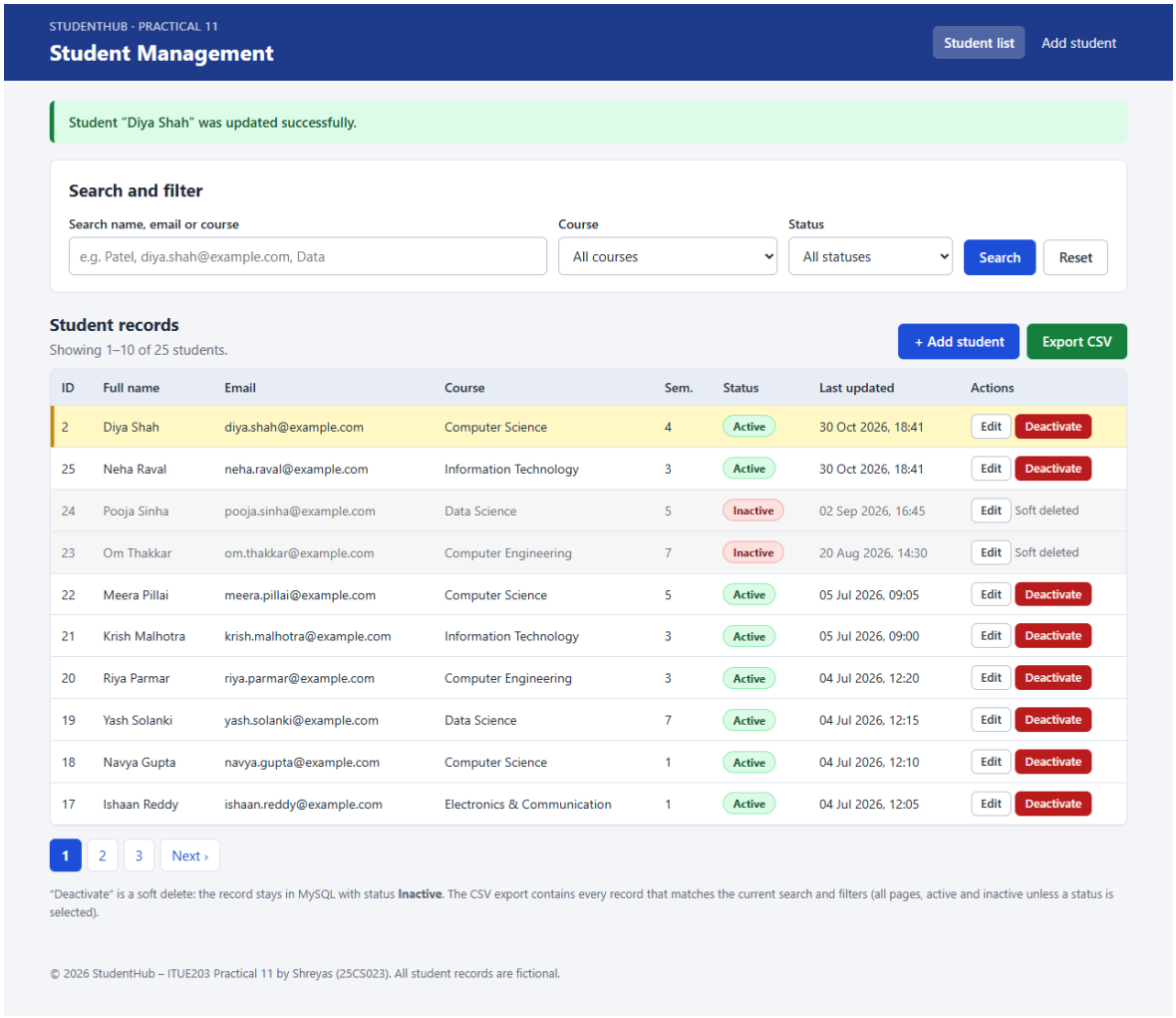


Figure 5: Successful update – message and the updated details at the top of the list (successful-update.png)

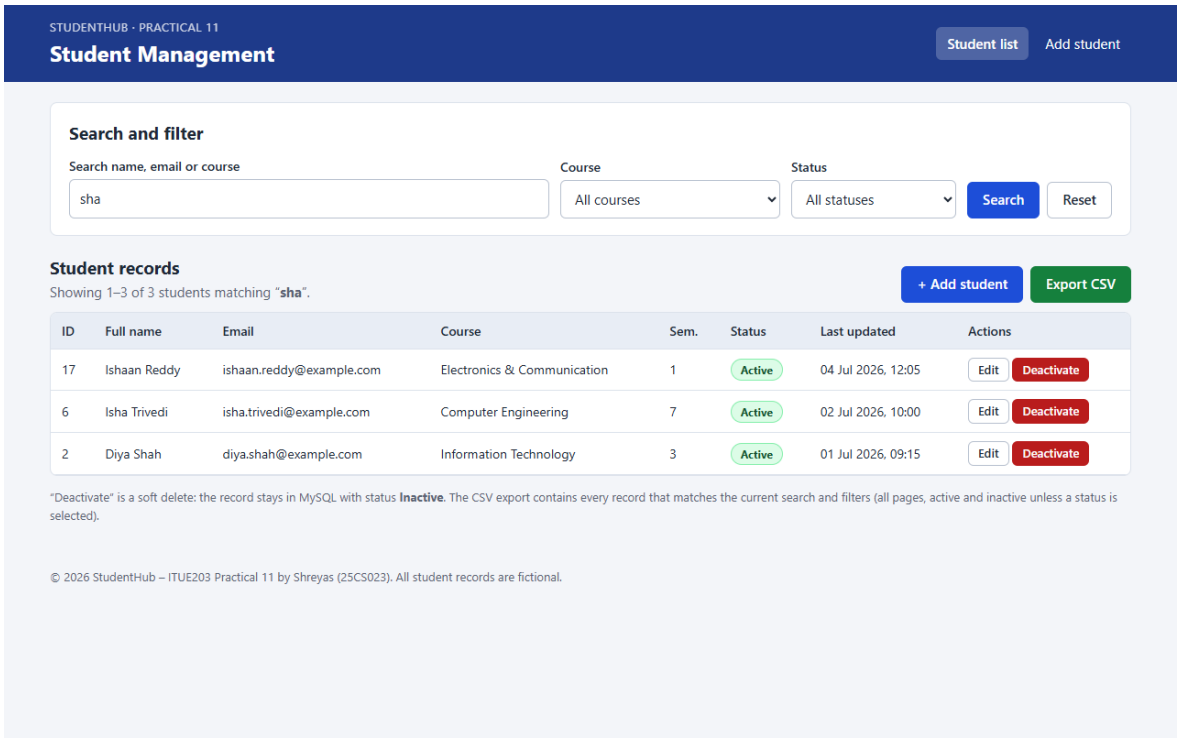


Figure 6: Search "sha" – matches Diya Shah, Isha Trivedi and Ishaan Reddy by name and email (search-result.png)

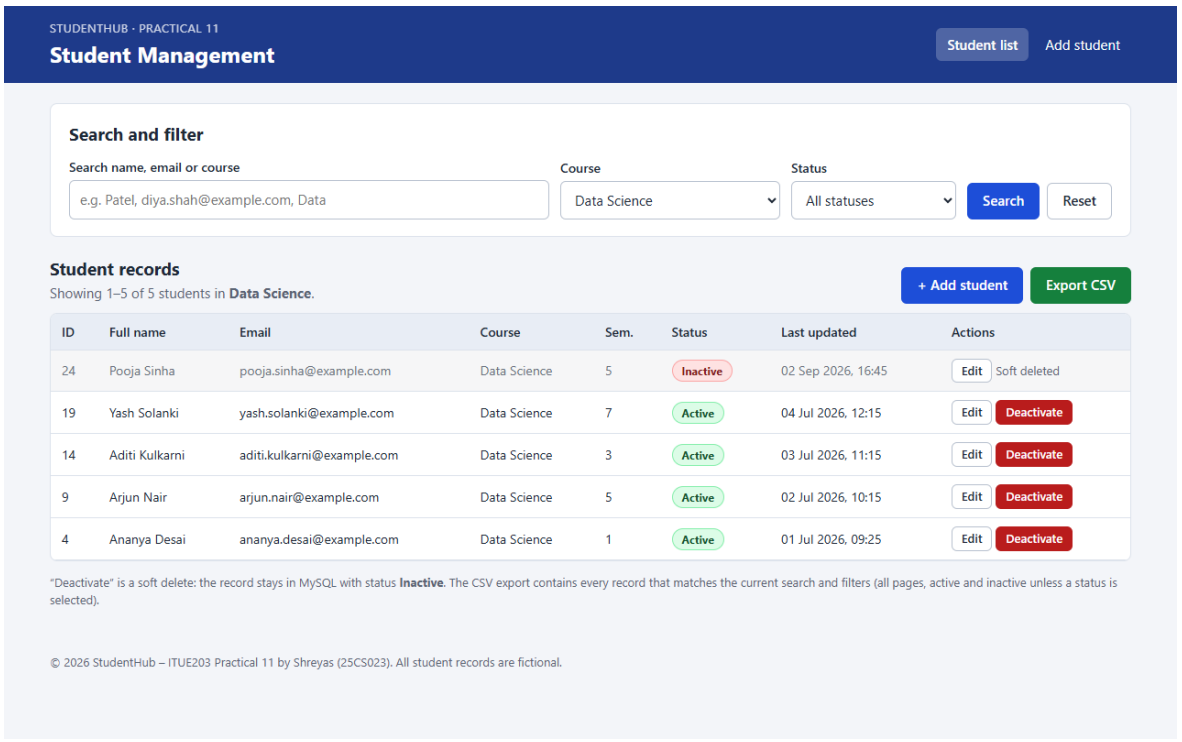


Figure 7: Course-wise filtering – only Data Science students (active and inactive) (course-filter.png)

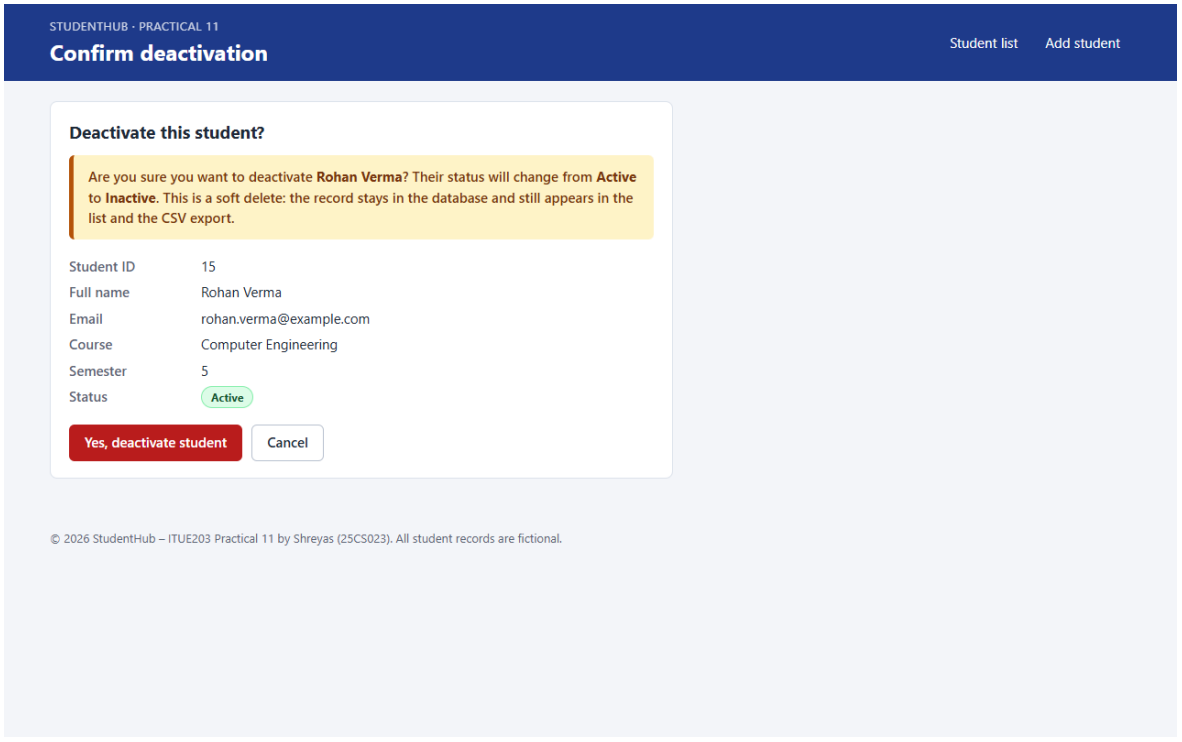


Figure 8: Confirmation before the soft delete of Rohan Verma (ID 15) (delete-confirmation.png)

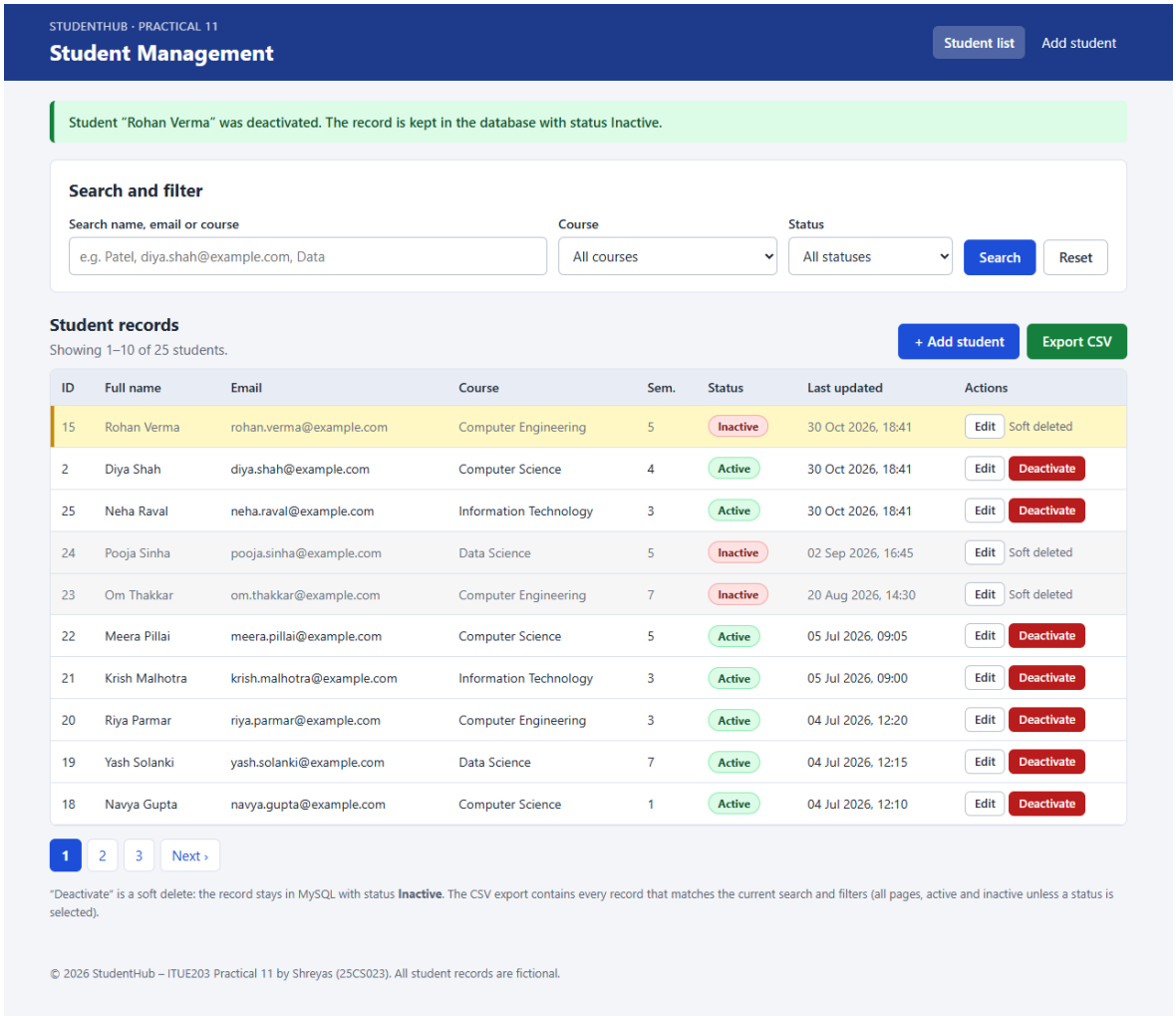


Figure 9: Successful soft delete – Rohan Verma kept in the list with status Inactive (successful-soft-delete.png)

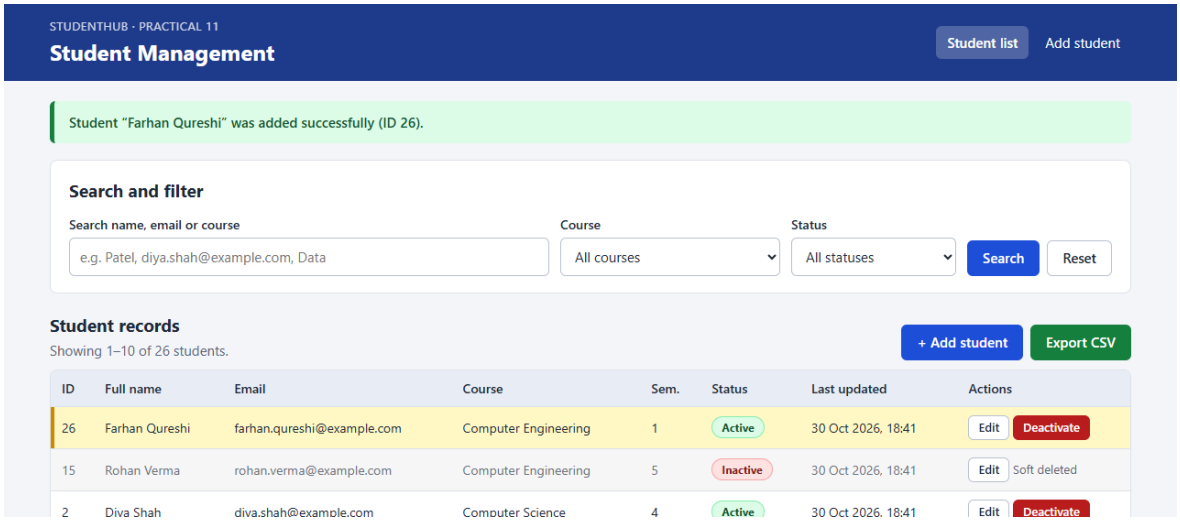


Figure 10: Success message after adding another student (Farhan Qureshi, ID 26) (success-message.png)

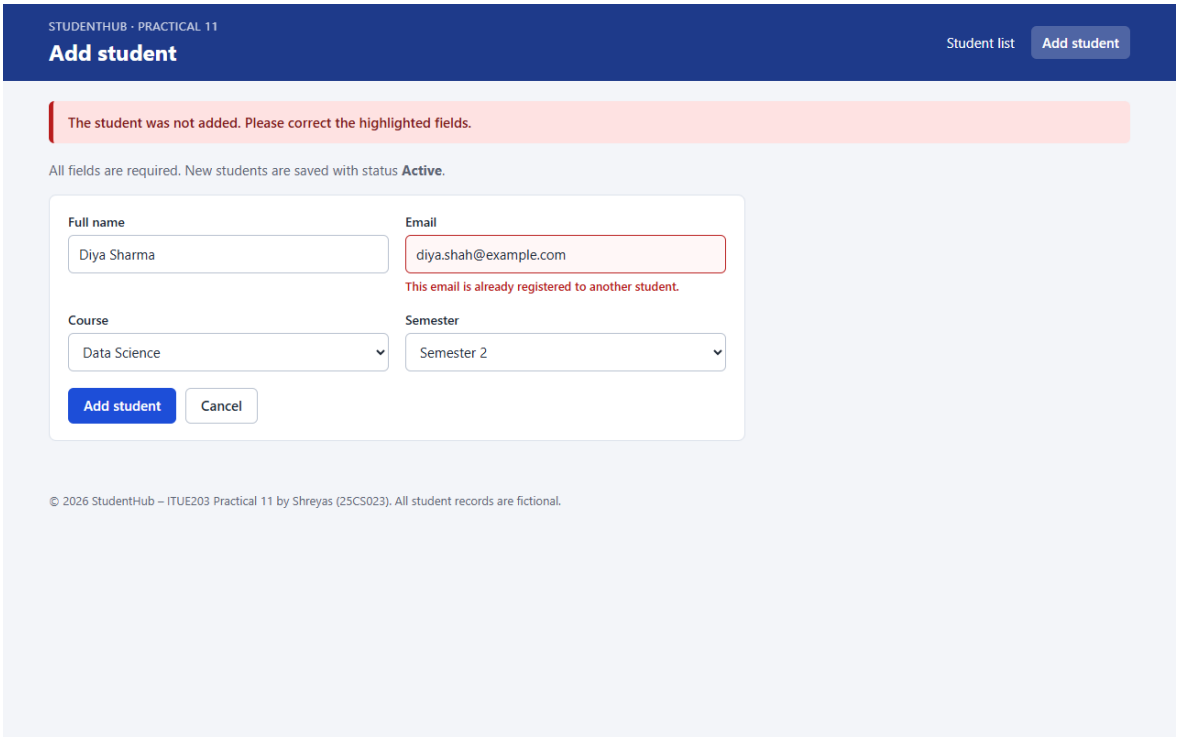


Figure 11: Error message – the server refuses a duplicate email (diya.shah@example.com) (error-message.png)

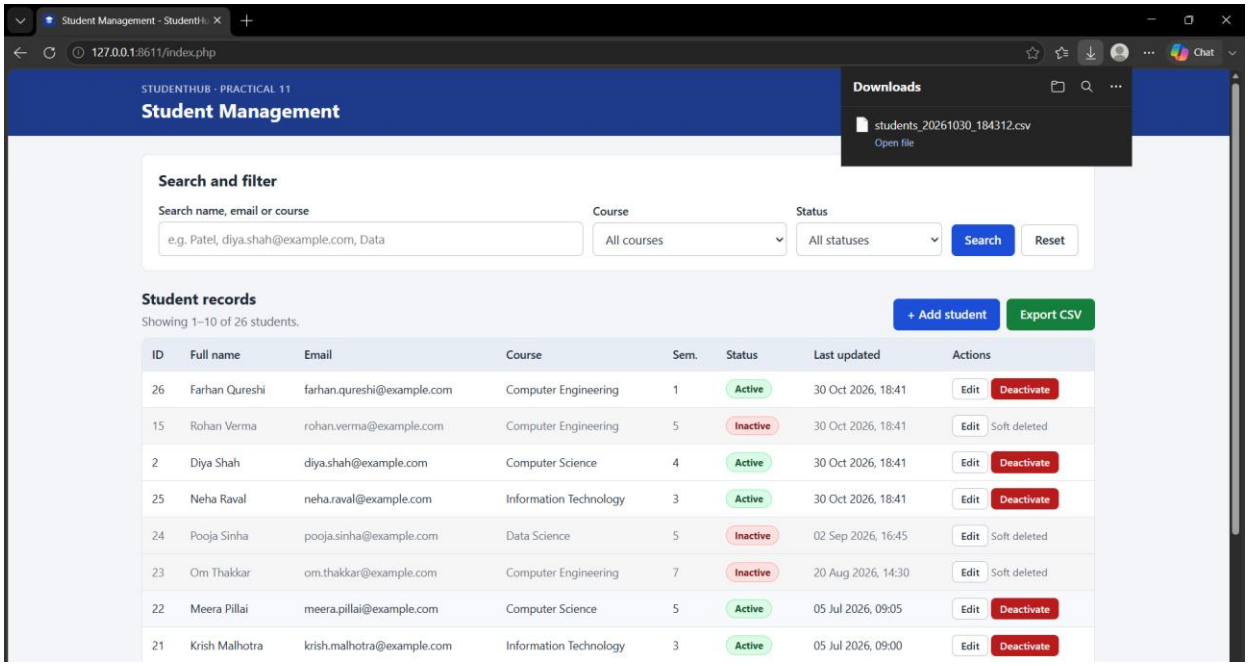
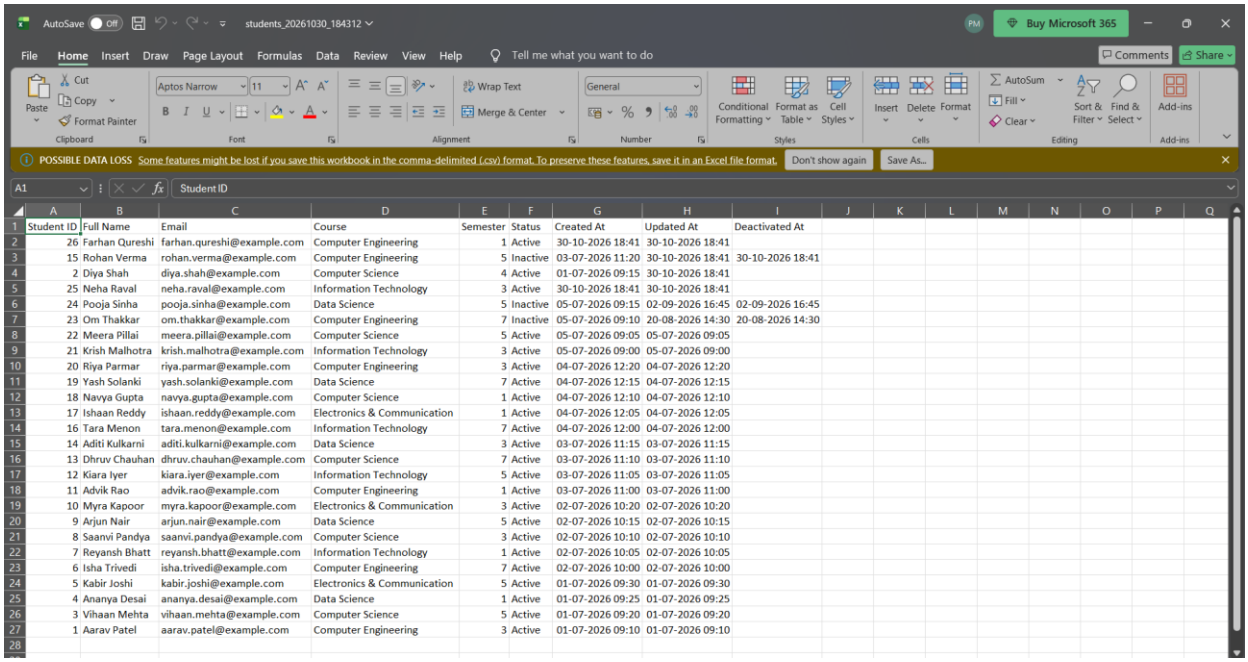


Figure 12: CSV export – Edge's Downloads panel shows students_20261030_184312.csv after clicking Export CSV (csv-export.png)



Student ID	Full Name	Email	Course	Semester	Status	Created At	Updated At	Deactivated At
26	Farhan Qureshi	farhan.qureshi@example.com	Computer Engineering	1	Active	30-10-2026 18:41	30-10-2026 18:41	
15	Rohan Verma	rohan.verma@example.com	Computer Engineering	5	Inactive	03-07-2026 11:20	30-10-2026 18:41	30-10-2026 18:41
2	Diya Shah	diya.shah@example.com	Computer Science	4	Active	01-07-2026 09:15	30-10-2026 18:41	
25	Neha Raval	neha.raval@example.com	Information Technology	3	Active	30-10-2026 18:41	30-10-2026 18:41	
24	Pooja Sinha	pooja.sinha@example.com	Data Science	5	Inactive	05-07-2026 09:15	02-09-2026 16:45	02-09-2026 16:45
23	Om Thakkar	om.thakkar@example.com	Computer Engineering	7	Inactive	05-07-2026 09:10	20-08-2026 14:30	20-08-2026 14:30
22	Meera Pillai	meera.pillai@example.com	Computer Science	5	Active	05-07-2026 09:05	05-07-2026 09:05	
21	Krish Malhotra	krish.malhotra@example.com	Information Technology	3	Active	05-07-2026 09:00	05-07-2026 09:00	
20	Riya Parmar	riya.parmar@example.com	Computer Engineering	3	Active	04-07-2026 12:20	04-07-2026 12:20	
19	Yash Solanki	yash.solanki@example.com	Data Science	7	Active	04-07-2026 12:15	04-07-2026 12:15	
18	Navya Gupta	navya.gupta@example.com	Computer Science	1	Active	04-07-2026 12:10	04-07-2026 12:10	
17	Ishaan Reddy	ishaan.reddy@example.com	Electronics & Communication	1	Active	04-07-2026 12:05	04-07-2026 12:05	
16	Tara Menon	tara.menon@example.com	Information Technology	7	Active	04-07-2026 12:00	04-07-2026 12:00	
14	Aditi Kulkarni	aditi.kulkarni@example.com	Data Science	3	Active	03-07-2026 11:15	03-07-2026 11:15	
13	Dhruv Chauhan	dhruv.chauhan@example.com	Computer Science	7	Active	03-07-2026 11:10	03-07-2026 11:10	
12	Kiara Iyer	kiara.iyer@example.com	Information Technology	5	Active	03-07-2026 11:05	03-07-2026 11:05	
11	Advik Rao	advik.rao@example.com	Computer Engineering	1	Active	03-07-2026 11:00	03-07-2026 11:00	
10	Myra Kapoor	myra.kapoor@example.com	Electronics & Communication	3	Active	02-07-2026 10:20	02-07-2026 10:20	
9	Arjun Nair	arjun.nair@example.com	Data Science	5	Active	02-07-2026 10:15	02-07-2026 10:15	
8	Saanvi Pandya	saanvi.pandya@example.com	Computer Science	3	Active	02-07-2026 10:10	02-07-2026 10:10	
7	Reyansh Bhatt	reyansh.bhatt@example.com	Information Technology	1	Active	02-07-2026 10:05	02-07-2026 10:05	
6	Isha Trivedi	isha.trivedi@example.com	Computer Engineering	7	Active	02-07-2026 10:00	02-07-2026 10:00	
5	Kabir Joshi	kabir.joshi@example.com	Electronics & Communication	5	Active	01-07-2026 09:30	01-07-2026 09:30	
4	Ananya Desai	ananya.desai@example.com	Data Science	1	Active	01-07-2026 09:25	01-07-2026 09:25	
3	Vihaan Mehta	vihaan.mehta@example.com	Computer Science	5	Active	01-07-2026 09:20	01-07-2026 09:20	
1	Aarav Patel	aarav.patel@example.com	Computer Engineering	3	Active	01-07-2026 09:10	01-07-2026 09:10	

Figure 13: The exported CSV opened in Microsoft Excel – 26 students, 9 columns, inactive rows with Deactivated At (csv-opened-in-excel.png)

7. Test Cases and Results

`tests/test_app.py` (Python standard library) creates the disposable database `studenthub_prac11_test` from the SQL dump, starts two PHP servers (normal, and with MySQL unreachable), sends real HTTP requests and checks the pages and the database. It removes the test database at the end; `studenthub_prac11` is not used.

Result: 106 of 106 automated checks passed, 0 failed. `php -l` found no syntax errors in the 9 PHP files. The browser test in Microsoft Edge passed **13 of 13** checks (including: the browser blocks an empty form, no JavaScript errors).

ID	Test case	Expected result	Result
TC01	Database connection	page loads (HTTP 200) with data from `studenthub_prac11_test`	Pass
TC02	Seed data	24 records, 22 active, 2 inactive; UNIQUE email and CHECK constraints work	Pass
TC03	Student list display	table with ID, name, email, course, semester, status badge, last updated, actions; "Showing 1–10 of 24 students"	Pass
TC04	Successful addition	303 → list, "Student “Neha Raval” was added successfully", row first and highlighted, email stored in lower case, status active	Pass
TC05	Empty-field validation	HTTP 422, an error under each of the 4 fields, nothing inserted; in Edge the browser blocks the empty form	Pass
TC06	Invalid email	"Enter a valid email address…", typed values kept, nothing inserted	Pass
TC07	Duplicate email	"This email is already registered to another student.", nothing inserted	Pass
TC08	Tampered values	all three refused by the server	Pass
TC09	Successful update	"Student “Diya Shah” was updated successfully.", MySQL row changed, row first	Pass
TC10	Update errors	error messages, nothing saved; unknown ID → "That student record was not found."	Pass
TC11	Soft-delete confirmation	confirmation page "Are you sure you want to deactivate Rohan Verma?"; GET changes nothing	Pass
TC12	Successful soft delete	"…was deactivated. The record is kept in the database with status Inactive."	Pass
TC13	Status changed to inactive	`status='inactive'`, `deleted_at` set; red *Inactive* badge, no Deactivate button	Pass

TC14	Inactive record remains	row count unchanged; no `DELETE FROM` anywhere in the code	Pass
TC15	Soft delete twice / unknown ID	"…is already inactive." / "record was not found", nothing changed	Pass
TC16	Search by name	only Aarav Patel	Pass
TC17	Search by email	only Kiara Iyer	Pass
TC18	Search by course	all Data Science students (case-insensitive)	Pass
TC19	Search edge cases	"No students found"; `%` searched literally	Pass
TC20	Course filter	only that course; & escaped as `&`	Pass
TC21	Combined filters	result equals the matching SQL count	Pass
TC22	Pagination	10 + 10 + 4 rows, each student once; out-of-range pages clamped; page links keep the filter	Pass
TC23	Success and error messages	green success or red error message; shown once only	Pass
TC24	CSV export	`text/csv` attachment `students_YYYYMMDD_HHMMSS.csv`, UTF-8 BOM, CRLF, header + one row per student, 9 columns	Pass
TC25	CSV contents and filters	only matching rows; inactive rows have Status *Inactive* and *Deactivated At*	Pass
TC26	CSV formula injection	exported as `'=HYPERLINK(…)`, `'+1+1`, … (text, not formulas)	Pass
TC27	Excel compatibility	27 rows × 9 columns, readable, & and dates correct (`csv-opened-in-excel.png`)	Pass
TC28	CSRF validation	refused with "The form could not be processed…", nothing changed	Pass
TC29	Duplicate submission	first accepted, second refused (one record)	Pass
TC30	SQL injection	treated as text / refused; no extra rows, no change, table intact; `O'Brien` stored literally	Pass
TC31	XSS output escaping	shown escaped (`<…>`) in the list, edit form, confirmation page and re-shown form	Pass
TC32	Raw database errors hidden	HTTP 503 "The student database is not available right now", no technical details; real error in the PHP log	Pass
TC33	Protected files	HTTP 403	Pass

Checks per area (from tests/test_output.txt): static 4, database 8, list 6, pagination 6, add 5, validation 9, update 7, soft delete 9, search/filter 10, CSV 9, CSRF/duplicates 7, SQL injection 7, XSS 5, protected files 9, server log 1, MySQL unreachable + cleanup 4 = 106.

8. Problems Encountered and Fixes

- **Sorting within one second:** the list shows the most recently changed record first using `updated_at`, which has 1-second precision. In one test run an insert and an update happened in the same second and the updated student was not first. The order is still deterministic (ties by ID); the test now waits one second and the limitation is documented.
- **Test expectations:** the search “sha” also (correctly) matches *Isha* Trivedi, and a soft delete of an inactive student answers with a redirect; two checks were corrected to expect the real, correct behaviour.
- **Error highlighting:** the red border for invalid fields was overridden by a more specific CSS rule; the selector was fixed.
- **CSV download evidence:** with Playwright the download is intercepted and Edge shows a random file name, so Figure 12 was captured from a normal Edge window instead.
- **router.php:** a shell here-document turned `'\\'` into `'\'`; `php -l` reported the parse error and it was fixed.

9. Conclusion

The Student Management application fulfils all required functions: records are listed in a table, students are added and updated with validation and clear messages, the list can be searched by name, email and course, filtered by course and paged, and students are soft-deleted only after confirmation – their status becomes

inactive while the record stays in MySQL. The CSV export respects the selected filters, includes inactive records and opens correctly in Excel. Prepared statements, output escaping, one-time form tokens and hidden error details protect the application, and all 106 automated checks and 13 browser checks passed. Possible extensions are a login (as in Practical 10) and a button to reactivate inactive students.

10. References

- PHP Manual – MySQLi prepared statements: https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php
- PHP Manual – htmlspecialchars: https://www.php.net/manual/en/function.htmlspecialchars.php
- PHP Manual – fputcsv: https://www.php.net/manual/en/function.fputcsv.php
- MariaDB Knowledge Base – CHECK constraints: https://mariadb.com/kb/en/constraint/
- OWASP – CSV Injection: https://owasp.org/www-community/attacks/CSV_Injection
- OWASP – Cross-Site Request Forgery Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- OWASP – SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- Project repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 (folder Practical-11)